

ללמוד Angular על ידי בניית Compass

מסלול מבוסס-פרויקט: מהשורה הראשונה שלך
ב-TypeScript ועד לאפליקציית Angular פרוסה

מהדורה ראשונה, 2026

הוכן לשימוש אישי.

הקדמה

אם מעולם לא בנית אפליקציית ווב, הספר הזה נכתב עבורך. הוא מניח שאתה יודע לקרוא אנגלית, להשתמש במחשב, ומוכן להקליד דברים בתוך עורך קוד ולראות מה קורה. הוא לא מניח שאתה יודע מהו פריימוורק, מהו TypeScript, או למה מישו יבחר מרצון להשתמש במשהו שנקרא "הזרקת תלויות". עד לפרק האחרון, כל המונחים האלה ירגישו לך ככלים שאתה שולח אליהם יד, לא כז'רגון שאתה סובל.

הספר נבנה סביב אפליקציה יחידה בשם **Compass** – מעקב אישי אחר משימות והרגלים. בפרק 4 תקליד `ng new compass` ותראה תבנית התחלתית ריקה מופיעה על המסך. כל פרק שיבוא אחר כך מוסיף יכולת אמיתית אחת ל-Compass. בסוף הספר, תהיה לך אפליקציה אמיתית: היא תאפשר לך להתחבר, לתכנן את השבוע שלך, לעקוב אחרי רצפים, לעבוד ללא חיבור לאינטרנט, לסנכרן כשתתחבר מחדש, ולפעול בדפדפן באינטרנט הפומבי.

ללמידה על ידי בנייה יש שני יתרונות שספרי "קריאה תחילה" בדרך כלל מוותרים עליהם. הראשון הוא שכל מושג נוחת איפשהו. כשאתה פוגש רעיון חדש – נניח, פיירט (interceptor) – אתה לא סתם קורא הגדרה; אתה כותב אחד, רואה איך הוא משנה את התנהגות האפליקציה שלך, ואז שובר אותו בכוונה כדי לראות מה

קורה. השני הוא שאתה מסיים עם משהו. Compass קטן מספיק כדי להחזיק בראש, ועשיר מספיק כדי להראות לחבר. הוא גם שלך: אין בספר חידה שהתשובה שלה חבויה בסוף.

איך הספר מאורגן

לספר יש חמישה חלקים ונספחים.

חלק ראשון – יסודות. ארבעה פרקים שמעבירים אותך מאפס לפרויקט Angular רץ. תפגוש את TypeScript, את התכונות המודרניות של JavaScript ש-Angular באמת משתמש בהן, ואת ה-CLI של Angular. לא תיגע כאן בקומפוננטה; המטרה היא שיהיה לך סדנה לפני שתרים פטיש.

חלק שני – קומפוננטות ותבניות. המכניקה המרכזית: מהי קומפוננטה, איך תבניות וקישורי נתונים עובדים, ואיך סיגנלים – הפרימיטיב הריאקטיבי המודרני של Angular – מאפשרים לך לתאר מצב משתנה ללא טקסים. בסוף חלק שני, Compass ייראה וירגיש כמו אפליקציה אמיתית.

חלק שלישי – שירותים והעולם שבחוץ. אפליקציות לא חיות בקומפוננטה אחת. חלק זה מכסה הזרקת תלויות, HTTP, RxJS וטפסים – ארבעת הדברים שאתה צריך כדי לדבר עם שרתים, לאסוף קלט ממשתמשים, ולשתף התנהגות בין חלקי האפליקציה בלי להעתיק ולהדביק.

חלק רביעי – בניית אפליקציה אמיתית. ניתוב, מצב שחורג מקומפוננטה אחת, דירקטיבות ופייפים מותאמים אישית, וביצועים.

כאן Compass גדל מ"עמוד אחד שעובד" ל"כמה עמודים שנשארים מהירים".

חלק חמישי – הפקה. בדיקות, רינדור בצד השרת (SSR), פריסה, והמערכת האקולוגית שאליה אתה מצטרף. כשתסיים את חלק חמישי, יהיה לך מופע של Compass באינטרנט, ותדע מה לחפש הלאה.

נספחים. מדריך התקנה למחשב, רשימת בדיקות ל-debugging, וסיור קצר באידיומים ישנים של Angular שתפגוש בקוד קיים. קרא אותם כשתצטרך; הם חומר עזר, לא רצף.

איך לקרוא את הספר

קרא אותו ליד המקלדת. לכל פרק יש קוד שכדאי להקליד – לא להדביק. ההקלדה עצמה עובדת: היא מאיטה אותך מספיק כדי לראות מה השתנה, ומאלצת אותך לשים לב לתחביר שהעתקה הייתה מחליקה מעליו. כשפרק מסתיים בסעיף תרגילים, עשה אותם לפני שתתחיל את הפרק הבא. הם לא רשות במובן שהפרק הבא מניח שעשית אותם; הם רשות במובן שאם תדלג עליהם, תאבד את הזיכרון השרירי שהספר מנסה לבנות.

הודעות מודגשות מופיעות לאורך הספר:

הערה. צד-סוגריים שימושי – הקשר מעמיק, מושג קשור, או פיסת היסטוריה.

שים לב. טעות נפוצה או התנהגות עדינה שתחסוך לך אחר צהריים בהמשך.

נסה זאת. ניסוי של דקה לשינוי קוד וראייה של התוצאה. עשה את אלה; הם זולים.

מה צריך

- מחשב נייד או שולחני עם macOS, Windows או Linux.
- כ-5GB של שטח דיסק פנוי (בעיקר עבור `node_modules`, שנפגוש בפרק 4).
- חיבור אינטרנט יציב בזמן העבודה על חלקים ראשון וחמישי; שאר הספר עובד ללא רשת.
- בערך 30 עד 50 שעות פרוסות על פני מספר ישיבות שנוח לך. יש קוראים שסיימו בשני סופי שבוע; יש שפרסו על פני שלושה חודשים. הספר לא אכפת לו.

אתה לא צריך ניסיון קודם ב-Vue, React, Angular או כל פריימוורק אחר. אתה לא צריך לדעת TypeScript; פרק 2 מלמד מספיק ממנו. אתה לא צריך לדעת איך שרתים עובדים; כשנצטרך שרת בחלק שלישי, נרים אותו יחד בכמה שורות של קוד.

מילה על גרסאות של Angular

Angular משחררת גרסה ראשית חדשה בערך כל שישה חודשים, אבל הרעיונות בספר הזה – קומפוננטות, סיגנלים, הזרקת תלויות, ניתוב, טפסים – יציבים ומרכזיים כבר שנים ויישארו כאלה. התחביר הספציפי מיועד ל-Angular 18 ואילך, שהיא הגרסה הראשונה שבה מבנה הזרימה החדש (`@if`, `@for`, `@switch`), סיגנלים, וקומפוננטות עצמאיות הם ברירת המחדל המומלצת. אם אתה קורא את זה כש-Angular נמצאת בגרסה 22 עד אז, רוב הקוד עדיין ירוץ בלי שינוי; מה שכן ישתנה יהיה שינוי שם, לא שינוי חשיבה.

הפוך את הדף. פרק 1 קצר – בלי קוד, רק מפה של איפה אנחנו ולאן אנחנו הולכים.

פרק 1. מהי Angular, ומתי כדאי לפנות אליה

בפרק הזה אין קוד. הוא מפה. לפני שתתקין דבר, יעזור לך לדעת מה אתה עומד להתקין, איזו בעיה זה פותר, ואיזה סוג של אפליקציה אתה עומד לבנות. קרא אותו פעם אחת, לאט. כל מה שיבוא מפרק 2 ואילך מניח שהמפה הזאת בראש שלך.

הצורה של אפליקציית ווב

דמיין, לרגע, אתר שאתה משתמש בו לעתים קרובות. אולי זה יומן, רשימת משימות, שירות סטרימינג של מוזיקה, או תיבת דואר. עכשיו דמיין מה הוא עושה כשאתה מתקשר איתו.

אתה לוחץ על הודעה והטקסט המלא שלה מופיע בחלונית מימין. אתה מקליד שאילתת חיפוש ותוצאות מתסננות תוך כדי הקלדה, בלי שהעמוד נטען מחדש. אתה מסמן משימה כבוצעה והיא עוברת לרשימת "הושלמו"; משתמש אחר, ביבשת אחרת, רואה את השינוי שנייה לאחר מכן. אתה גורר אירוע ביומן שלך והוא מצטרף לזמן חדש.

שום דבר מזה אינו עמוד. בשנים הראשונות של הווב, כל אחת מהאינטראקציות האלה הייתה מחייבת מסע הלוך ושוב לשרת: לחיצה, המתנה, צפייה בעמוד מתלבן, המתנה, ראיית עמוד חדש. אפליקציית ווב מודרנית עושה משהו שונה. היא נטענת פעם אחת, ואז שומרת

תמונה חיה של מה שהמשתמש מסתכל עליו, מעדכנת חלקים מהמסך כשמידע חדש מגיע וכשהמשתמש פועל. כשהיא צריכה מידע או רוצה לשמור משהו, היא מדברת עם השרת בשקט ברקע, בלי לשבש את התמונה.

לשמור על התמונה החיה מעודכנת קשה יותר ממה שזה נשמע. כל חלק על המסך תלוי בפיסת מידע כלשהי – מספר ההודעות שלא נקראו בסרגל הצד תלוי ברשימת ההודעות, מצב הכפתור "השב" תלוי בכך שהודעה נבחרה, הצבע של משימה תלוי אם היא באיחור. כשכל פיסת מידע משתנה, כל חלק של המסך שתלוי בה חייב להשתנות איתה. אם היית מנסה לכתוב את זה ידנית – כל כפתור שצריך להיכנס לפעולה ולצאת ממנה, כל מונה שצריך לגדול, כל רשימה שצריכה לרנדר מחדש – היית מבלה את רוב הזמן שלך בהדבקת דברים במקום לבנות משהו חדש. גם היית מכניס באגים בלי הפסקה: שוכח לעדכן מונה, שוכח להשבית כפתור, שוכח להסיר פריט ישן מרשימה.

פריימוורק הוא תוכנה שמטפלת ברוב ההדבקה הזאת בשבילך. Angular היא פריימוורק.

מה "פריימוורק" באמת אומר

אם כתבת תוכנית מחשב בעבר – אפילו קטנה – כתבת את הקוד שלך ובאיזשהו שלב ייבאת את הקוד של מישהו אחר כדי לעשות עבודה שלא רצית לעשות בעצמך. הקוד של אותו מישהו נקרא **ספרייה**. הקוד שלך אחראי; כשהוא צריך עזרה, הוא קורא לספרייה.

פריימוורק הוא ההפך. בפריימוורק, הקוד של הפריימוורק אחראי; הקוד שלך ממלא את החלקים שהפריימוורק לא יודע. משל שימושי: ספרייה היא פטיש, ואתה מחליט מתי להניף אותו. פריימוורק הוא סדנה, ואתה מחליט מה לבנות בתוך הסדנה – אבל הסדנה כבר בחרה את פריסת שולחן העבודה, את מיקום השקעים, ואת ארונות הכלים על הקיר. אתה מרוויח הרבה: הסדנה מאורגנת, כלים סטנדרטיים תמיד במקום שאתה מצפה שיהיו, ואנשים אחרים שעבדו באותו סוג של סדנה יכולים להתמצא בקוד שלך מייד. אתה גם מוותר על משהו: אם אתה רוצה לעקור את שולחן העבודה ולשים את ארונות הכלים על התקרה, הסדנה תילחם בך.

הבחירה הזאת היא הבחירה המרכזית שאתה עושה כשאתה בוחר פריימוורק. בתמורה לווייתור על גמישות מסוימת, אתה מקבל עקביות, קהילה, והרבה החלטות שאינך צריך יותר לעשות בעצמך. Angular נמצאת בצד ה"בעל דעה מוצקה" של הספקטרום. יש לה דעות על איך לארגן קוד, איך קומפוננטות מדברות זו עם זו, איך לטפל בתופעות לוואי, איך לכתוב בדיקות, ואיך לבנות להפקה. חלק מהחדשים בעניין מוצאים את זה מחניק בימים הראשונים ומשחרר לשארית הקריירה שלהם.

אז – מהי Angular?

Angular היא פריימוורק, שנעשה על ידי Google, לבניית אפליקציות שרצות בדפדפן ווב. במשפט אחד: **אתה מתאר את המסכים של האפליקציה שלך כעץ של חתיכות קטנות ועצמאיות שנקראות קומפוננטות, ו-Angular דואגת להפוך את התיאור הזה לפיקסלים,**

לשמור על הפיקסלים בסנכרון עם הנתונים שלך, ולשלוח את כל העסק לדפדפן.

בואו נפרק את המשפט הזה חלק אחר חלק, כי לכל שם עצם בו יהיה פרק משלו בהמשך.

עץ של קומפוננטות. לא תבנה את Compass כקובץ ענק אחד. תבנה אותו כעץ של חלקים קטנים: מעטפת ברמה העליונה, בתוכה סרגל צד ואזור מרכזי, בתוך האזור המרכזי כותרת ורשימה, בתוך הרשימה שורות רבות של משימות, וכן הלאה. כל חלק – כל קומפוננטה – אחראי לדבר אחד: איך הוא נראה, איך הוא מתנהג כשמתקשרים איתו, ואיך הוא חושף את עצמו לחלקים שסביבו. קומפוננטות בעץ מדברות עם ההורים והילדים שלהן דרך חיבורים מוגדרים היטב וחד-כיווניים. Angular קוראת לחיבורים האלה inputs ו-outputs; תפגוש אותם כראוי בפרק 8.

עצמאיות. ה-HTML, ה-CSS וההתנהגות של הקומפוננטה חיים ביחד. זהו היפוך מכוון של המודל הישן של פיתוח ווב, שבו לפרויקט הייתה תיקייה של קבצי HTML, תיקייה של קבצי CSS, ותיקייה של קבצי JavaScript, וכל פיצ'ר יחיד נגע בקובץ אחד בכל אחת. הטיעון של Angular, שנולד מעשור של צפייה באנשים בונים אפליקציות גדולות, הוא שפיצ'רים לעתים רחוקות מכבדים את הגבולות האלה – הסימון של כפתור, הסגנון שלו, ומטפל הלחיצה שלו הם פיצ'ר אחד – ושמועיל יותר לשמור אותם יחד ולהפריד בין פיצ'רים במקום.

Angular הופכת את התיאור שלך לפיקסלים. אתה מתאר איך המסך צריך להיראות בכל רגע נתון, בהינתן נתונים מסוימים. Angular מבינה מה בעצם לשים על המסך, ומה לשנות כשהנתונים משתנים. זה

עדין. בווּב הישן, כתבת קוד שאמר "כשהמשתמש לוחץ 'השב', הסר את כפתור ההשבה, הוסף כפתור שלה, שים textarea באמצע, הצב עליו את המיקוד". ב-Angular, אתה כותב: "כפתור ההשבה גלוי כשאין טיוטה; כפתור השליחה גלוי כשיש טיוטה; תוכן ה-textarea קשור לטקסט של הטיוטה". אתה מתאר את המצבים, לא את המעברים. Angular עושה את המעברים.

Angular שומרת על הפיקסלים בסנכרון עם הנתונים שלך. כי התיאור שלך אומר איזה חלקים של המסך תלויים באיזה חלקים של הנתונים, Angular יודעת איך לעדכן אותם כשהנתונים משתנים. Angular מודרנית עושה את זה עם מנגנון שנקרא סיגנלים, שתפגוש בפרק 7. סיגנל הוא פיסת נתונים שיודעת מי מסתכל עליה; כשהנתונים משתנים, כל מי שמסתכל עליהם מקבל התראה, ורק החלקים של המסך שקוראים אותם מרונדרים מחדש. זה נשמע כמו נחמדות ומתגלה כמהפכני: זה אומר שהאפליקציה שלך נשארת מהירה גם כשהיא גדלה, כי הפריימוורק אף פעם לא צריך לבדוק שוב את כל המסך אחרי כל שינוי.

Angular שולחת את כל העסק לדפדפן. אפליקציות ווב מודרניות עוברות שלב בנייה שהופך את קוד המקור שלך (הרבה קבצים בפורמט חיבור ידידותי) למספר קטן של קבצים, ממוקטמים בכבדות, מוכנים להישלח לדפדפן. Angular מגיעה עם כלי בנייה שמטפל בזה עבורך. בפרק 4 תריץ את `ng build` בפעם הראשונה ותראה מזה-בייטים של קוד המקור שלך דחוסים למאות בודדות של קילו-בייטים של פלט מוכן להפקה.

מה הופך את Angular ל-Angular

Angular היא לא הפריימוורק היחיד מסוגה. אם שמעת את השם Angular, כנראה שמעת גם את השמות של כמה אחרים. הספר הזה לא ישווה בין כולם לעומק – אתה לא צריך את ההשוואה הזאת כדי ללמוד Angular יותר משאתה צריך להשוות בין ספרדית לאיטלקית כדי ללמוד ספרדית – אבל הוגן לתת שם לתכונות שמבדילות את Angular מעמיתיה.

בעלת דעה מוצקה וכוללת סוללות. Angular מגיעה עם ניתוב, טפסים, HTTP, אנימציה, בדיקות, בינאום, ורינדור בצד השרת כבר בקופסה, כולם מעוצבים לעבוד יחד. פריימוורקים שנוקטים בגישה ההפוכה – שולחים ליבה קטנה ונותנים לך לבחור ספריות לכל השאר – נותנים לך יותר גמישות ומאלצים אותך לקבל יותר החלטות. ההימור של Angular הוא שלרוב ההחלטות האלה יש תשובות נכונות רוב הזמן, והיא עושה אותן עבורך כדי שתוכל להשקיע את תשומת הלב שלך באפליקציה שלך במקום זה.

TypeScript תחילה. Angular כתובה ב-TypeScript, וכל אפליקציית Angular נכתבת ב-TypeScript. TypeScript היא סופר-סט של JavaScript: זה JavaScript עם טיפוסים. אם זה נשמע מפחיד, אל דאגה – פרק 2 מלמד מספיק TypeScript כדי להיות פרודוקטיבי, והכלים של Angular גורמים לטיפוסים לעזור לך יותר משהם מפריעים. התמורה אמיתית: כשאתה משנה שם של שדה, העורך שלך משנה את שמו בכל השימושים שלו על פני האפליקציה כולה; כשאתה שובר את החוזה של קומפוננטה עם ההורה שלה, העורך שלך אומר לך לפני שאתה מרענן

את הדפדפן. באפליקציה גדולה, זה ההבדל בין קוד שאתה יכול להחזיק בראש לבין קוד שאתה נלחם בו.

הזרקת תלויות. ל-Angular יש מערכת מובנית לקומפוננטה אחת לומר "אני צריכה משהו שעושה X" ולקבל דבר מתאים בלי שתצטרך לדעת איך הוא נבנה. זה נשמע מופשט עכשיו; זה ירגיש מובן מאליו אחרי פרק 9. זו הסיבה הבודדת הגדולה ביותר לכך שאפליקציות Angular נוטות להיות בדיקות, כי "תן לי גרסה מזויפת של הדבר שמדבר עם השרת" מתגלה כשורת קוד אחת.

סיגנלים כפרימיטיב הריאקטיבי. רוב ההיסטוריה של Angular, המודל הריאקטיבי שלה נבנה על ספרייה שנקראת RxJS, שמייצגת שינויים לאורך זמן בזרמים נצפים. RxJS חזקה ועדיין חשובה – תפגוש אותה בפרק 11 – אבל צוק הלמידה היה תלול והטקסים היו כבדים למצב יומיומי. גרסאות אחרונות של Angular הציגו סיגנלים, פרימיטיב קל יותר שמרגיש כמו משתנים רגילים אבל מעדכן את המסך אוטומטית כשהם משתנים. סיגנלים הם איך שתנהל את רוב המצב בספר הזה; RxJS הוא איך שתטפל בזרמים שנפרסים במהותם לאורך זמן, כמו תגובות HTTP וזרמי אירועים.

CLI מונחה קונבנציות. Angular מגיעה עם כלי שורת פקודה, ng, שמייצר תבניות של פרויקטים, יוצר קומפוננטות ושירותים חדשים, מריץ את האפליקציה שלך, מריץ את הבדיקות שלך, ובונה להפקה. זה הכלי שמסיר את רוב הטקסים של ההקמה שפעם היו שער לפיתוח ווב. בפרק 4 תשתמש בו כדי ליצור את Compass בפקודה אחת.

מתי Angular היא בחירה טובה – ומתי לא

Angular היא לא הבחירה הנכונה לכל פרויקט בווב, וחלק מהלמידה של הפריימוורק הוא לדעת איפה הוא זורח.

Angular היא במיטבה כשהאפליקציה היא כזאת שאתה מצפה לעבוד עליה לאורך זמן, עם צוות, עם מורכבות עסקית אמיתית: דשבורדים, פאנלים של מנהלים, כלים פנימיים, פורטלים ללקוחות, אפליקציות בנקאות ובריאות ומסחר אלקטרוני שיצמחו פיצ'רים ברבעון במשך שנים. בהקשרים האלה, המבנה בעל הדעה של Angular, יסוד ה-TypeScript, וסיפור הבדיקות משתלמים פי כמה. מהנדסים חדשים יכולים להצטרף ולהתמצא; refactor בטוחים כי הקומפיילר תופס שבירות של חוזים; ניתן להוסיף פיצ'רים בלי לשבור בטעות שלושה פיצ'רים אחרים.

Angular היא כלי כבד יותר ממה שאתה צריך לעמוד שיווקי בודד, אתר נחיתה, בלוג, או הדגמה אינטראקטיבית חד-פעמית. עבור אלה, HTML פשוט – או מחולל אתרים סטטי קל משקל – יתחיל אותך מהר יותר, ישלח פחות קוד למשתמשים שלך, ויהיה קל יותר להעביר הלאה. לבחור ב-Angular לאתר שיווקי של חמישה עמודים זה כמו לשכור מחסן כדי להכין כריך. אתה יכול לעשות את זה; זה יעבוד; לא כדאי לך.

Angular היא לא הבחירה הטובה ביותר למנועי משחקים, ויזואליזציות כבדות, או אפליקציות שהמשטח העיקרי שלהן הוא canvas או סצנת WebGL. Angular לא יודעת ולא אכפת לה מה נראה פיקסל של ה-canvas שלך; היית משתמש בה רק לממשק שסביב

ה-canvas, וגם אז פריימוורק קל יותר או קוד רגיל אולי היה מתאים יותר.

Compass, האפליקציה שאתה עומד לבנות, יושבת במרכז מקום הכיף של Angular: כמה מסכים, נתונים אמיתיים, שרת מתמיד, טפסים, אימות, ורשימת פיז'רים שתגדל לאורך הפרקים. הוא מייצג את סוג האפליקציה שצוותי Angular בונים כל יום.

מה Compass יהיה, בסוף

עד לפרק האחרון, Compass יהיה אפליקציה אמיתית, אם גם קטנה. הנה תצוגה מקדימה כדי שתדע לאן הדרך מובילה.

תפתח את Compass בדפדפן שלך ותראה מסך התחברות. תתחבר עם כתובת מייל; קישור אישור יגיע; תלחץ עליו, ו-Compass יזכור אותך במכשיר הזה. תגיע לדשבורד שמראה את המשימות של היום ואת ההרגלים שאתה עוקב אחריהם השבוע.

תוסיף משימה בלחיצה על כפתור, מילוי טופס, ולחיצה על שמור. היא תופיע מיד. אם האינטרנט שלך כבוי, היא עדיין תופיע מיד, מסומנת באיקון קטן שאומר "עדיין לא סונכרן", וכשהאינטרנט יחזור היא תישמר בשקט ברקע. תגרור משימות בין "היום", "השבוע הזה", ו"מאוחר יותר". תסמן הרגלים כבוצעו בלחיצה; מונה רצפים ליד כל הרגל יתעדכן.

תנווט לעמוד סטטיסטיקות ותראה תרשים של השלמות ההרגלים שלך על פני שלושים הימים האחרונים. תלחץ על הרגל כדי לראות את

ההיסטוריה שלו. תבקר בעמוד הגדרות שבו תוכל לשנות את שם התצוגה שלך ולבחור ערכת נושא.

כל אחד מהפיצ'רים האלה הוא מקום בספר. מסך ההתחברות ואישור המייל חיים בפרקי הניתוב והטפסים. ההתנהגות של offline-first חיה בפרקי ה-HTTP והמצב. התרשים חי בפרק הקומפוננטות והתבניות, ושוב בפרק הביצועים כשנעשה אותו מהיר מספיק כדי לא להיות משנה בטלפון. הבדיקות שתכתוב לכל זה חיות בחלק חמישי.

תסיים את הספר עם אפליקציה שקטנה מספיק כדי להחזיק בראש ועשירה מספיק כדי להצדיק כל רעיון ש-Angular מציעה.

מה בא הלאה

שלושת הפרקים הבאים מכינים אותך להקליד `ng new compass`. הם יוצאי דופן עבור ספר "בונים אפליקציה" כי הם מכילים כמעט אפס Angular. זה במכוון. Angular יושבת על גבי TypeScript, TypeScript יושבת על גבי JavaScript מודרני, ושניהם יושבים על גבי סט של כלים – מנהל חבילות, סביבת ריצה, עורך. דילוג עליהם, כמו שהרבה ספרים עושים, מייצר קוראים שיכולים להעתיק דוגמאות של Angular אבל שקופאים בפעם הראשונה שהודעת שגיאה מזכירה מונח שהספר מעולם לא הגדיר.

פרק 2 מכסה את TypeScript. לא את כולו – TypeScript הוא שפה גדולה עם סט טיפוסים אפשריים גדול עוד יותר – רק את החלקים שבאמת תפגוש כשתכתוב Angular. פרק 3 מכסה את התכונות של JavaScript המודרני ש-Angular מסתמכת עליהן: `destructuring`, `spread`,

פונקציות חץ, מודולים, promises ו- `await / async`. פרק 4 יעביר אותך דרך התקנת Node והתקנת Angular CLI, הרצת `ng new` `compass`, וסיור איטי על כל קובץ שה-CLI מייצר כדי שכלום בפרויקט החדש שלך לא ייראה מסתורי.

בסוף חלק ראשון, תהיה לך אפליקציית Angular רצה על המסך שלך ותבין כל חלק ממה שיצר אותה. אז, בחלק שני, נתחיל לבנות.

תרגילים

1. פתח אפליקציית ווב שאתה משתמש בה לעתים קרובות – לקוח מייל, שירות מוזיקה, כלי לניהול פרויקטים, כל דבר עם ממשק עשיר. שים לב מה קורה כשאתה לוחץ מסביב. נסה לזהות שלוש אינטראקציות שהיו דורשות טעינה מלאה של עמוד באתר בסגנון ישן ובמקום זה מעדכנות רק פיסה של המסך. רשום אותן.

2. במילים שלך, בשלושה או ארבעה משפטים, תאר את ההבדל בין ספרייה לפריימוורק למישהו שמעולם לא כתב קוד. אל תשתמש במילים ספרייה או פריימוורק בתשובה שלך. אם אתה יכול להסביר את החליפין בלעדיהן, אתה מבין אותם.

3. חשוב על רעיון אפליקציה שאולי תרצה לבנות יום אחד. הוא לא חייב להיות מקורי או טוב; קופסת מתכונים אישית, יומן תרגול מוזיקה, או תזכורת ליום הולדת זה בסדר. שאל את עצמך: האם האפליקציה הזאת נמצאת במקום הכיף של Angular, או שמשהו קל יותר ישירות אותה יותר טוב? רשום

את התשובה שלך ואת ההיגיון שלך. בעוד כמה חודשים,
חזור וראה אם אתה עדיין מסכים.

פרק 2. TypeScript מספיק ל-Angular

הפרק הזה הוא ידע עבודה של TypeScript, שנכתב לאנשים שלא השתמשו בה קודם ולא רוצים להפוך לחוקרי TypeScript במשרה מלאה. בסוף תוכל לקרוא כל ביטוי TypeScript שהספר משתמש בו, ולכתוב משלך בלי לנחש. לא תדע – ואל תשאף לדעת – כל מה ש-TypeScript יכולה לעשות. מערכת הטיפוסים שלה עמוקה מספיק כדי שיהיה לה מדף ספרים משלה; אנחנו לא מבקרים במדף הזה היום.

הדוגמאות בפרק הזה מונעות על ידי Compass. בכל פעם שנציג תכונה חדשה של TypeScript, נראה אותה עם צורה שתהיה לך אחר כך באפליקציה שלך: משימה, הרגל, משתמש, אירוע השלמה. כך, עד שנתחיל לכתוב קומפוננטות ב-Angular בפרק 5, הטיפוסים כבר ירגישו מוכרים.

למה טיפוסים בכלל

אם כתבת סקריפט קטן בשפה כמו Python או JavaScript רגיל, כנראה כתבת משהו כזה:

```
function totalMinutes(sessions) {
  let sum = 0;
  for (const s of sessions) sum += s.duration;
  return sum;
}
```

זה עובד. זה גם לא נותן לך שום עזרה. במקום אחר בתוכנית, קולגה מעביר לך מערך של מחרוזות בטעות, או אובייקט שמאיית את השדה `durationMinutes` במקום `duration`. הפונקציה עדיין רצה; היא מייצרת `NaN` או `"0[object Object]"` או משהו אחר סוריאליסטי, ועד שאתה שם לב, אתה צד אחר המקור של המוזרות בפלט.

TypeScript מוסיפה כמות קטנה של טקסים כדי למנוע את הציד הזה. אתה מוסיף הערה למה שפונקציה מצפה; הקומפיילר של TypeScript בודק כל מקום שהפונקציה נקראת בו; אם קורא מפר את החוזה, אתה מגלה מייד, לפני שהקוד רץ. בעורך, זה מתגלה כקו אדום גלי מתחת לשורה הבעייתית והודעה ספציפית על מה שגוי. בבנייה, זה עוצר את הבנייה.

אותה פונקציה ב-TypeScript:

```
interface Session {
  duration: number;
}

function totalMinutes(sessions: Session[]): number {
  let sum = 0;
  for (const s of sessions) sum += s.duration;
  return sum;
}
```

עכשיו הקומפיילר יודע: הפונקציה מקבלת מערך של דברים שיש להם שדה `duration` שהוא מספר, ומחזירה מספר. תעביר לה משהו אחר ו-TypeScript תתנגד. אין שום קסם בזה – הקומפיילר פשוט בודק ששמות וצורות מסכימים – אבל התמורה בקוד גדול יותר מקובץ אחד משמעותית. Angular בנויה להישען על התמורה הזאת.

הרצת TypeScript, בפסקה אחת

TypeScript היא לא משהו שדפדפנים מריצים ישירות. כלי שנקרא הקומפיילר של TypeScript (`tsc`) קורא קבצי `.ts`, בודק את הטיפוסים, ומייצר קבצי JavaScript רגילים שדפדפן (או Node.js) יכול להריץ. בפרויקטי Angular, אתה כמעט אף פעם לא מפעיל את `tsc` בעצמך; ה-CLI של Angular מריץ אותו כחלק מ-`ng serve` ו-`ng build`. לפרק הזה, אם אתה רוצה לנסות TypeScript בבידוד, השתמש ב-TypeScript Playground באתר typescriptlang.org/play – הוא נותן לך להקליד TypeScript משמאל ולראות את ה-JavaScript המהודר מימין, בלי התקנה. מפרק 4 והלאה, ה-CLI לוקח פיקוד.

הפרימיטיבים

הטיפוסים הבסיסיים של TypeScript הם `number`, `string`, `boolean`, `null` ו-`undefined`. יש אחרים, אבל אלה אלה שתקליד מאה פעמים ביום.

```
let title: string = "Buy milk";
let priority: number = 2;
let done: boolean = false;
let due: string | null = null; // עוד על "|" בעוד רגע
```

TypeScript חכמה בהסקת טיפוסים כשאתה מציב ערך בנקודת ההצהרה:

```
let title = "Buy milk"; // מוסק string
```

אתה לא צריך לכתוב `string` : כשהערך כבר אומר ל-TypeScript מה המשתנה מחזיק. בפועל, אתה מוסיף הערות טיפוסים על פרמטרים של פונקציות וטיפוסי החזרה, על צורות של אובייקטים, ועל דברים שאתה עומד להקצות אליהם אחר כך; אתה משאיר את כל השאר להסקה. קוד שמוסיף הערות לכל `let` טריוויאלי הוא קוד מלא רעש.

הערה. `null` ו-`undefined` שונים ב-TypeScript, כמו שהם ב-Angular. JavaScript. נוטה להעדיף `null` עבור "אנחנו יודעים שאין כאן ערך" ולשמור את `undefined` עבור "הערך מעולם לא הוצב". שדה שהוא אופציונלי הוא כזה שאולי הוא `undefined`; שדה שהוא `nullable` הוא כזה שקיים אבל אולי הוא `null`. זו הבחנה עדינה שתהיה חשובה בפרק 12 כשנדבר על טפסים.

מערכים וטאפלים

הטיפוס של מערך הוא `Type[]`, או במקביל `Array<Type>`:

```
const priorities: number[] = [1, 2, 3];
const tags: string[] = ["home", "work"];
```

מערכים ריקים צריכים הערה כי אין מה להסיק מהם:

```
const completed: string[] = []; // בסדר
const uh = [];                  // שאתה לא רוצה, any-מוסק כ
```

טאפלים הם מערכים באורך קבוע וטיפוסים לפי מיקום. תשתמש בהם לעתים רחוקות בקוד אפליקציה:

```
const point: [number, number] = [42, 7];
```

טיפוסי אובייקט עם interface

הלחם והחמאה של TypeScript באפליקציית Angular הוא תיאור הצורה

של אובייקטים. TypeScript מציעה שתי דרכים לעשות את זה – `interface` ו-`type` – ולצורות של אובייקטים הן כמעט חליפיות. השתמש ב-`interface` לצורות של אובייקטים; השתמש ב-`type` לאיחודים, לחיתוכים, ולדברים שהם לא סתם אובייקט.

הנה הטיפוס המרכזי של Compass, כפי שיופיע בפרק 5:

```
interface Task {
  id: string;
  title: string;
  done: boolean;
  createdAt: string; // ISO 8601 תאריך
  dueDate: string | null;
  tags: string[];
}
```

ערך שטוען שהוא `Task` חייב להכיל את כל שישה השדות האלה, עם הטיפוסים האלה. שדות נוספים הם שגיאה (ברוב ההקשרים). שדות חסרים הם שגיאה. טיפוסים שגויים הם שגיאה. הצורה הזאת הופכת לחוזה שכל חלק של האפליקציה יכול להסתמך עליו.

מאפיינים אופציונליים, `readonly`, וחתימות אינדקס

תפגוש שלוש הערות שמשנות את התנהגות השדה.

```
interface Habit {
  id: string;
  name: string;
  color?: string; // undefined עשוי להיות
  readonly createdAt: string; // לא ניתן להקצות מחדש אחרי בנייה
  metadata: { [key: string]: string }; // string→string מפה שרירותית
}
```

`color?: string` אומר שהשדה אופציונלי; ערך שמקיים את `Habit` לא חייב לכלול אותו, ואם הוא שם הוא עשוי להיות `string` או `undefined`. `readonly` מונע הקצאה מחדש דרך הטיפוס הזה – מועיל לשדות בסגנון `id` שאסור לשנות במקום. הצורה האחרונה, `{ [key: string]: string }`, נקראת חתימת אינדקס; היא

אומרת "כל מפתח מסוג `string` ממופה לערך מסוג `string`" ומועילה למילונים פתוחים.

איחודי טיפוסים וטיפוסים ליטרליים

טיפוס איחוד הוא ערך שיכול להיות אחד מכמה טיפוסים. כבר ראית אחד: `string | null`. איחודים הם "או" מוגבל: הערך הזה הוא או מחרוזת או הערך `null`.

איחודים של מחרוזות ליטרליות הם אחד הדפוסים הכי שימושיים של TypeScript:

```
type Status = "todo" | "doing" | "done" | "archived";

interface Task {
  id: string;
  title: string;
  status: Status;
}
```

עכשיו `status` לא יכול להיות מחרוזת שרירותית; הוא חייב להיות בדיוק אחד מארבעת אלה. נסה לכתוב `task.status = "in-progress"` ו-TypeScript תסרב. כך אתה אוכף סט סגור של אפשרויות — `enum` של עני, וברוב המקרים הבחירה הטובה יותר.

שים לב. ל-TypeScript יש מילת מפתח `enum`, והשניים הראשונות של Angular השתמשו בה בכבדות. הפרקטיקה המודרנית היא להעדיף איחודי מחרוזות ליטרליות. `enum` נהדק לאובייקטים JavaScript של זמן ריצה, שמגדילים את ה-bundle שלך; איחודי טיפוסים נהדקים לכולם (הם קיימים רק בזמן בדיקת הטיפוסים). פרק 5 משתמש באיחודים לכל אורכו.

צמצום טיפוסים

אם למשתנה יש טיפוס איחוד, TypeScript תסיק צמצום שלו בתוך ענף של קוד שהיא יכולה לומר בו מה הערך באמת:

```
function greeting(name: string | null): string {
  if (name === null) {
    return "Hello, stranger.";
  }
  // name יודעת ש string, לא string | null.
  return `Hello, ${name}.`;
}
```

שלושת דפוסי הצמצום הנפוצים ביותר:

- `typeof x === "string"` מצמצם את `x` ל-`string`.
- `x === null` או `x === undefined` מצמצמים דרך `if` ים.
- `"field" in x` מצמצם לתת-הקבוצה של איחוד שיש לה את השדה.

• `x instanceof Foo` מצמצם ל-`Foo`.

צמצום הוא אחת התכונות שגורמות ל-TypeScript להרגיש כאילו היא עוזרת במקום נלחמת. אם TypeScript אי פעם אומרת "אולי `undefined`" ואתה יודע שלא, התשובה כמעט תמיד היא לבדוק עם `if`, לא לעקוף את הבדיקה.

פונקציות

טיפוסי פונקציה מוסיפים הערות לפרמטרים, ואופציונלית לטיפוסי החזרה:

```
function isOverdue(task: Task, now: Date): boolean {
  if (task.dueDate === null) return false;
  return new Date(task.dueDate) < now;
}
```

פונקציות חץ עובדות אותו הדבר:

```
const isOverdue = (task: Task, now: Date): boolean => {
  if (task.dueDate === null) return false;
  return new Date(task.dueDate) < now;
};
```

אם TypeScript יכולה להסיק את טיפוס ההחזרה מהגוף, אתה יכול להשמיט את ההערה — אבל זו פרקטיקה טובה להוסיף הערות לטיפוסי החזרה בפונקציות מיוצאות, כי זה מכריח אותך לחשוב על מה שפונקציה בעצם מבטיחה להחזיר.

פרמטרים של פונקציה יכולים להיות אופציונליים או שיהיו להם ברירות מחדל:

```
function label(task: Task, prefix?: string): string {
    return `${prefix ?? ""}${task.title}`;
}

function paginate<T>(items: T[], page: number, size = 20): T[] {
    return items.slice(page * size, (page + 1) * size);
}
```

הדוגמה השנייה משתמשת ב-גנרי, שנסתכל עליו הבא.

גנריים, ברוגע

גנרי הוא פרמטר טיפוס – מציין מקום ל"טיפוס כלשהו שנמלא באתר הקריאה". השימוש הקלאסי הוא פונקציה שעובדת על כל מערך בלי לאבד מידע טיפוס:

```
function first<T>(items: T[]): T | undefined {
    return items[0];
}

first([1, 2, 3]); // T הוא number; מחזיר number | undefined
first(["a", "b"]); // T הוא string; מחזיר string | undefined
```

השם **T** הוא קונבנציה. השתמש באותיות בודדות לפרמטרים מאוד כלליים (**V**, **K**, **U**, **T**), או בשמות תיאוריים כשבהירות עוזרת (**<Payload>**, **<Item>**).

תראה גנריים כל הזמן ב-Angular. `signal<Task[]>([])` יוצר סיגנל שמחזיק מערך של `Task`. `HttpClient.get<Task[]>('/api/tasks')` אומר לקומפיילר איזה צורה לצפות בחזרה מהשרת. בכל מקרה, הגנרי הוא התשובה ל"איזה טיפוס נמצא בתוך הדבר?".

אל תכתוב פונקציות גנריות משלך כשאתה לומד. פנה אליהן בפעם השלישית או הרביעית שאתה מעתיק ומדביק פונקציה ורק משנה את הטיפוסים שלה. עד אז, פונקציות עם טיפוסים רגילים הן כמעט תמיד הכלי הנכון.

הסקת טיפוסים היא חברה שלך

TypeScript אגרסיבית בהסקת טיפוסים, וכדאי לך לתת לה. שקול:

```
const tasks = tasksFromServer();           // מוסק כ-Task[]
const titles = tasks.map(t => t.title);     // מוסק כ-string[]
const activeCount = tasks.filter(t => !t.done).length; // מוסק כ-number
```

לא היו צריכות הערות — TypeScript צעדה דרך כל השרשרת והבינה. הכלל הכללי: הוסף הערות לכניסות וליציאות של הפונקציות הציבוריות שלך, ותן ל-TypeScript להסיק את כל השאר.

any, unknown, ופתחי המילוט הכנים

לפעמים אתה באמת לא יודע את הטיפוס של ערך – פשוט ניתחת קצת JSON מהחוט, או אתה קורא לספרייה ישנה בלי טיפוסים. ל-TypeScript יש שני פתחי מילוט:

- **any** מכבה בדיקת טיפוסים לערך הזה. הכל מותר. זה מסוכן; אם אתה כותב **any** אתה חותם ומוותר על כל היתרון של שימוש ב-TypeScript עבור הפיסת מידע הזאת.
- **unknown** היא החלופה הבטוחה. ערך של טיפוס **unknown** יכול להיות כל דבר, אבל אתה לא יכול לעשות שום דבר איתו עד שתצמצם אותו לטיפוס ספציפי קודם.

```
function parseTask(raw: unknown): Task {
  if (typeof raw !== "object" || raw === null) {
    throw new Error("Not an object");
  }
  const r = raw as { id?: unknown; title?: unknown; /* ... */ };
  if (typeof r.id !== "string" || typeof r.title !== "string") {
    throw new Error("Missing required fields");
  }
  // בדיקות נוספות...
  return { id: r.id, title: r.title, done: false, createdAt: "", dueDate: null };
}
```

פנה ל-**unknown** בגבול של האפליקציה שלך – ניתוח JSON, קריאת **localStorage**, טיפול בספריות בלי טיפוסים. פנה ל-**any** כמעט אף פעם.

דקורטורים, בקצרה

Angular מסתמכת בכבדות על דקורטורים – פונקציות שנכתבות עם קידומת `@` שמצמידות מטא-דאטה למחלקה או לחברים שלה. תראה אותם כל הזמן:

```
@Component({
  selector: "app-task-row",
  template: `<div>{{ task.title }}</div>`,
})
export class TaskRow {
  @Input() task!: Task;
}
```

לעת עתה, דע רק זאת: `@Component`, `@Input`, `@Output`, וחבריהם הם איך שאתה אומר ל-Angular "המחלקה הזאת היא קומפוננטה" או "השדה הזה הוא input מהורה". פרק 5 יסביר מה כל אחד עושה. תפקידה של TypeScript הוא פשוט לאפשר את התחביר; המשמעות היא של Angular.

הערה. דקורטורים סטנדרטים רשמית ב-JavaScript מודרני

עכשיו, אבל Angular עדיין משתמשת בטעם הניסיוני הישן יותר כי הוא נושא יותר מטא-דאטה ש-Angular מסתמכת עליה. ה-CLI של Angular מגדיר את הפרויקט שלך כך שזה פשוט יעבוד; אתה אף פעם לא צריך לחשוב על זה. אם אי פעם תראה `experimentalDecorators` ב-`tsconfig.json`, זו הסיבה.

האופרטורים ! ו- ?

שני סימנים קטנים ייחודיים ל-TypeScript יופיעו לעתים קרובות:

- `x!` הוא טענת non-null. הוא אומר לקומפיילר "סמוך עליי, זה לא null או undefined כאן". השתמש בו במידה; כל ! הוא מקום שבו יכולה להופיע שגיאת זמן ריצה שמערכת הטיפוסים לא מזהירה עליה. בקוד קומפוננטה, `@Input()` `task!: Task` משתמש ב-! כדי להבטיח "Angular תקצה את זה לפני שאשתמש בו", וזאת הבטחה ש-Angular מקיימת – אבל רק אחרי בניית הקומפוננטה, לא במהלכה.
- `x?.y` הוא אופרטור שרשור אופציונלי (שהוא בעצם JavaScript רגיל). הוא מחזיר `undefined` בקצר-מסלול אם `x` הוא `null` או `undefined`, במקום לזרוק. `"default" ?? x?.y` משלב אותו עם אופרטור `nullish` coalescing שנפגוש בפרק 3.

מה דילגנו עליו במכוון

ל-TypeScript יש טיפוסים מותנים, טיפוסים ממופים, טיפוס `template literal`, `infer`, איחודים מובחנים במלוא הכלליות שלהם, ומאה כלים אחרים. תיתקל בחלקם, בעיקר בהודעות שגיאה, ותפתה לצלול לחר הארנב הזה. התנגד. התכונות בפרק הזה הן אלה שתשתמש בהן כל יום ב-Angular. השאר זמין כשתצטרך אותו.

מה בא הלאה

פרק 3 מכסה את התכונות של JavaScript המודרני ש-Angular מניחה שאתה יודע: מודולים, פונקציות חץ, spread, destructuring ו-rest, await / async, ושיטות המערך שתישען עליהן ללא הפסקה. אם כבר כתבת JavaScript עדכני, אתה יכול לסקור אותו במהירות; אם לא, קח את זה לאט – פרק 4 יניח כל רעיון מפרק 3.

תרגילים

1. כתוב `Habit` interface עם השדות האלה: `id` (string), `name` (string), `frequency` (אחד מ- "daily", "weekly", "custom"), `color` (string אופציונלי), `createdAt` (string מסוג `readonly`). אז כתוב פונקציה `describe(habit: Habit): string` שמחזירה משהו כמו `"Read (daily)"`.

2. בהינתן `{ ok: true; value: T } type Result<T> = {`, כתוב פונקציה `{ ok: false; error: string }`, | `unwrap<T>(r: Result<T>): T` שמחזירה את הערך אם `ok` הוא `true` וזורקת אחרת. נסה לכתוב את זה עם `as`.

3. ב-Playground (typescriptlang.org/play), הדבק אובייקט בצורה `{ title: "test", priority: "high" }` ונסה להקצות אותו למשתנה בטיפוס `{ title: string; priority: number }`. קרא את הודעת השגיאה בקפידה. אחר כך תקן את האובייקט כך

שיעבור בדיקת טיפוסים. הבנת פורמט השגיאה הזה היא מיומנות שכדאי שתהיה.

פרק 3. JavaScript מודרני שבאמת תשתמש בו

Angular היא TypeScript מבחוץ ו-JavaScript מבפנים. כל מה שהקומפיילר מייצר, הדפדפן מריץ. אם קטע של תחביר JavaScript ייראה לא מוכר בדוגמת Angular בהמשך הספר, הסיכוי הוא שהיא אחת מהתכונות בפרק הזה. יש עוד עשרות שלא נכסה – שפת JavaScript המודרנית גדולה מאוד – אבל תת-הקבוצה כאן היא בערך מאה אחוז ממה שקוד יישום Angular באמת משתמש בו.

אם כתבת JavaScript עדכני קודם, אתה יכול לקרוא את הפרק הזה כרשימת בדיקה ולעבור במהירות מעל מה שאתה כבר יודע. אם לא, תכנן לנסות כל דוגמה בקונסולת הפיתוח של דפדפן (פתח את הדפדפן שלך, לחץ F12, לחץ על "Console") כדי שהתחביר יגיע לאצבעות שלך.

מודולים: `import` ו-`export`

כל קובץ בפרויקט JavaScript או TypeScript מודרני הוא מודול. מודול הוא קובץ שיכול לייצא דברים (פונקציות, מחלקות, קבועים, טיפוסים) לקבצים אחרים לשימוש, ולייבא דברים ממודולים אחרים. זה המנגנון שבאמצעותו האפליקציה שלך מפוצלת לקבצים בלי להם namespace גלובלי.

יש שני טעמים של ייצוא:

```
// tasks.ts

// ייצוא בשם: מייצא סמל תחת שמו שלו.
export interface Task {
  id: string;
  title: string;
}

export function isOverdue(task: Task): boolean { /* ... */ return false; }

// ייצוא ברירת מחדל: אחד לקובץ, מיובא בלי סוגריים מסולסלים.
export default class TaskService {
  // ...
}
```

ובמקביל, שני טעמים של ייבוא:

```
// app.ts

import TaskService from "./tasks"; // ברירת מחדל
import { Task, isOverdue } from "./tasks"; // בשם
import * as tasks from "./tasks"; // namespace הכול, עם
```

מדריך הסגנון של Angular, והספר הזה, מעדיפים ייצוא בשם כמעט תמיד. ייצוא ברירת מחדל חוסך זוג סוגריים אבל עולה בבהירות: השם של ייצוא ברירת מחדל באתר הייבוא הוא מה שהמייבא מקליד, כך ש-`import Foo from "./bar"` ו-`import Baz from "./bar"` מתייחסים לאותו דבר. ייצוא בשם שומר על השמות ישרים לרוחב הקוד.

נתיבי ייבוא שמתחילים ב-`./` או `../` הם יחסיים – הם מצביעים על קובץ אחר בפרויקט שלך. נתיבים שמתחילים בכל דבר אחר הם

מזהים חשופים ומתמייחסים לחבילות מותקנות (@angular/core , rxjs , וכן הלאה).

שים לב. אל תשים את הסיומת `.ts` בייבואים שלך (`from "./"` `tasks.ts`). כלי הבנייה של Angular מטפל בסיומת עבורך. בסביבות JavaScript אחרות – בעיקר Node.js במצב ESM – אתה כן צריך את הסיומת. Angular הוא המקרה הראשון, לא השני.

פונקציות חץ

פונקציית חץ היא דרך קצרה יותר לכתוב פונקציה, עם הבדל סמנטי חשוב אחד מפונקציה רגילה: אין לה `this` משלה.

```
// טופס ארוך
const isDone = function(task) { return task.done; };

// חץ, גוף בלוק
const isDone = (task) => { return task.done; };

// return חץ, גוף ביטוי – בלי סוגריים, בלי מילת מפתח
const isDone = task => task.done;

// מספר פרמטרים דורש סוגריים
const compare = (a, b) => a.priority - b.priority;
```

היעדר `this` משלה חשוב כשאתה מעביר פונקציית חץ כ-`callback`:

```

class TaskService {
  tasks: Task[] = [];

  loadTasks() {
    fetch("/api/tasks")
      .then(res => res.json())
      .then(data => {
        // `this` כאן עדיין מתייחס למופע של TaskService,
        // loadTasks-כי פונקציית החץ יורשת אותו מ
        this.tasks = data;
      });
  }
}

```

אם ה-callback הפנימי היה `function` רגיל, `this` בתוכו היה `undefined` (במצב strict) או האובייקט הגלובלי – מקור באגים שהציק ל-JavaScript עשור לפני שחצים הגיעו. בקוד Angular, כמעט תמיד תרצה חצים ל-callbacks.

Template literals

מחרוזות בגרשי קונטרס-אקצנט יכולות להכיל ביטויים אינטרפולציה ולהתפרש על פני מספר שורות:

```

const name = "Anna";
const greeting = `Hello, ${name}. Today is ${new Date().toLocaleString}`;

```

הן טובות באופן מוחלט מקישור מחרוזות לכל דבר ארוך יותר מטריויאלי. תבניות Angular עצמן הן template literals כשאתה כותב תבניות inline:

```
@Component({
  template: `
    <h1>Welcome, {{ user.name }}</h1>
    <p>You have {{ taskCount() }} tasks today.</p>
  `,
})
```

ה- `{{ }}` בפנים הוא תחביר התבנית של Angular, לא של JavaScript `$.{{ }}`. הם חיים יחד כי הם אף פעם לא חופפים: הליטרל החיצוני הוא JavaScript, מוערך פעם אחת כשמחלקת הקומפוננטה מוגדרת; ה- `{{ }}` הוא של Angular, מוערך מחדש כשהנתונים משתנים.

Destructuring

Destructuring שולף שדות מאובייקט (או אלמנטים ממערך) לתוך משתנים בשמות בביטוי יחיד.

```
const task = { id: "t1", title: "Buy milk", done: false };

// במקום:
const id = task.id;
const title = task.title;

// אתה יכול לכתוב:
const { id, title } = task;
```

אתה יכול לתת שם חדש בדרך החוצה ולספק ברירות מחדל:

```
const { id: taskId, title, done = false } = task;
```

Destructuring עובד גם בפרמטרים של פונקציה:

```
function label({ title, done }: Task): string {  
  return done ? `[x] ${title}` : `[ ] ${title}`;  
}  
  
label({ id: "t1", title: "Buy milk", done: false, /* ... */ });
```

Destructuring של מערכים משתמש באותו רעיון עם סוגריים מרובעים:

```
const [first, second, ...rest] = [1, 2, 3, 4, 5];  
// first = 1, second = 2, rest = [3, 4, 5]
```

ה-`...rest` בשורה האחרונה הוא אופרטור `spread/rest`, שנסתכל עליו הבא.

Spread-1 rest

לאופרטור שלוש-הנקודות `...` יש שני שימושים קשורים.

`Spread` מרחיב את האלמנטים של מערך (או את המאפיינים של אובייקט) `inline`:

```
const priorities = [1, 2, 3];  
const withZero = [0, ...priorities];    // [0, 1, 2, 3]  
  
const base = { title: "Untitled", done: false };  
const withId = { ...base, id: "t1" };    // { title, done, id }
```

`Spread` של אובייקט הוא איך שאתה יוצר עותק רדוד של אובייקט, ואיך שאתה מייצר עותק ששונה בלי לשנות את המקור:

```
const task: Task = { /* ... */ };
const updated = { ...task, done: true };
// task.done עדיין מה שהיה; updated.done הוא true.
```

הדפוס הזה של "אל תשנה; ייצר אובייקט חדש" הוא הדפוס של קוד ריאקטיבי מודרני ב-Angular. כשנגיע לסיגנלים בפרק 7, תראה אותו כל הזמן.

Rest אוסף את האלמנטים הנותרים של מערך או את הפרמטרים הנותרים של פונקציה למשתנה יחיד:

```
function sum(first: number, ...others: number[]): number {
  return others.reduce((acc, n) => acc + n, first);
}

sum(1, 2, 3, 4); // 10
```

אותן שלוש נקודות, כיוון הפוך: spread מרחיב, rest אוסף.

פרמטרים ברירת מחדל

לפרמטרים יכולות להיות ברירות מחדל, וברירות המחדל יכולות להתייחס לפרמטרים קודמים:

```
function paginate(items: Task[], page: number, size: number = 20): Task[] {
  return items.slice(page * size, (page + 1) * size);
}

paginate(all, 0); // size = 20 משתמש ב-
paginate(all, 0, 50); // size = 50 משתמש ב-
```

פרמטרי ברירת מחדל נכנסים לפעולה כשהארגומנט הוא `undefined` – העברת `null` לא מפעילה את ברירת המחדל. שווה לזכור את זה בפעם הראשונה שזה מפתיע אותך.

שרשור אופציונלי ו-nullish coalescing

שני אופרטורים שהחליפו בשקט הרבה שרשראות `if` הגנתיות.

שרשור אופציונלי – `?.` – מחזיר `undefined` בקצר-מסלול אם הצד השמאלי הוא `null` או `undefined`, במקום לזרוק:

```
const city = user?.address?.city;
// שווה ל:
const city2 = user && user.address ? user.address.city : undefined;
```

Nullish coalescing – `??` – מחזיר את הצד הימני אם הצד השמאלי הוא `null` או `undefined`, אחרת את הצד השמאלי:

```
const display = user.nickname ?? user.name ?? "Anonymous";
```

ההבחנה החשובה מ-`||` (שאולי ראית משמש באופן דומה) היא ש-`??` נופל דרך רק על `null` ו-`undefined`, לא על ערכים אחרים. `10 || 0` נותן `10`; `10 ?? 0` נותן `0`. ב-`Compass`, `3 ?? task.priority` הוא "ברירת מחדל בטוחה של עדיפות ל-3 אם לא מוגדרת", בעוד `3 || task.priority` היה שוגה ומעדכן עדיפות מכוונת של `0`.

Promises

Promise הוא ערך שיהיה קיים בעתיד. כל פעולה שלוקחת זמן – בקשת רשת, קריאת קובץ, טיימר – מחזירה **Promise**. **Promise** נמצא תמיד באחד משלושה מצבים: פתוח (עדיין ממתין), ממומש (הפיק ערך), או נדחה (הפיק שגיאה).

אתה מצמיד **callbacks** עם **.then()** ו- **.catch()**:

```
fetch("/api/tasks")
  .then(res => res.json())
  .then(tasks => console.log(tasks))
  .catch(err => console.error("Something went wrong:", err));
```

כל **.then** מקבל את הערך שהשלב הקודם החזיר. אם שלב כלשהו זורק או מחזיר **Promise** שנדחה, השליטה קופצת ל- **.catch**. הקרוב ביותר.

סגנון ה"שרשרת של **thens**" הזה היה פעם הדרך הסטנדרטית לכתוב קוד אסינכרוני. הוא עובד, ומנגנון ה-HTTP הבסיסי של **Angular** עדיין משתמש ב-**Promises** במקומות, אבל הארגונומיה מסורבלת ברגע שהשרשרת גדלה. קוד מודרני משתמש ב- **await / async** במקום.

await / async

async מסמן פונקציה כמחזירה **Promise**, ומאפשר לך להשתמש ב- **await** בתוכה. **await** משהה את הפונקציה בביטוי הזה עד שה-**Promise** שממתין נפתר, ואז מתחדש עם הערך שלו.

```
async function loadTasks(): Promise<Task[]> {  
  const res = await fetch("/api/tasks");  
  const tasks: Task[] = await res.json();  
  return tasks;  
}
```

// בהמשך:

```
const tasks = await loadTasks();
```

שגיאות ב-Promises שממתינים יוצאות כחריגות שנזרקות, כך שאתה יכול להשתמש ב-`catch` / `try` רגיל:

```
try {  
  const tasks = await loadTasks();  
} catch (err) {  
  console.error("Could not load tasks:", err);  
}
```

`await` / `async` היא דרך נחמדה יותר לכתוב את אותו החישוב ששרשרת של `.then()` ים מבטאת – אבל זה אותו החישוב. שום דבר לא רץ סינכרונית. `await` עדיין מוותר על שליטה לדפדפן בין שלבים. זה תחביר סוכר, וזה טעים.

הערה. Angular משתמשת לפעמים ב-Promises, לפעמים ב-Observables (טיפוס RxJS), ויותר ויותר בסיגנלים למצב ריאקטיבי. פרק 11 פורק את ההבדל בין Promises ל-Observables – לעת עתה, החשוב הוא שתבין Promises, כי כל השאר בעולם האסינכרוני בונה עליהם או מחליף אותם.

שיטות מערך שתשתמש בהן כל הזמן

שיטות המערך של JavaScript המודרני הן ארגז הכלים של קוד יישום יומיומי. תישען עליהן בכל יום.

```
const tasks: Task[] = /* ... */;

// map: הפוך כל אלמנט
const titles = tasks.map(t => t.title);

// filter: שמור את האלמנטים שתואמים
const active = tasks.filter(t => !t.done);

// find: undefined קבל את האלמנט הראשון שתואם, או
const buyMilk = tasks.find(t => t.title === "Buy milk");

// some / every: בדוק את המערך כמכלול
const anyOverdue = tasks.some(isOverdue);
const allDone = tasks.every(t => t.done);

// reduce: קפל את המערך לערך יחיד
const total = tasks.reduce((sum, t) => sum + (t.done ? 1 : 0), 0);

// includes: האם המערך מכיל את הערך?
const hasWork = tags.includes("work");

// sort: משנה! בדרך כלל תרצה להעתיק קודם
const byPriority = [...tasks].sort((a, b) => a.priority - b.priority);
```

שלוש אזהרות לגבי `sort` וחבריה:

1. `sort`, `reverse`, ו-`splice` משנים את המערך במקום.

אם אתה רוצה מערך חדש ממוין בלי להפריע למקורי, פזר

אותו קודם: `[...tasks].sort(...)`.

2. `sort` בלי פונקציית השוואה מכפה אלמנטים למחרוזות, שזה מקור באגים אגדי. מיונים מספריים דורשים פונקציית השוואה: `.sort((a, b) => a - b)`.
3. `reduce` חזקה ומופרזת בשימוש. אם אתה מוצא את עצמך כותב `reduce` שיכול להיות לולאת `for` עם שני משתנים, לולאת `for` בדרך כלל בהירה יותר.

לולאות `for`

יש לך שלוש דרכים עיקריות לעבור באיטרציה:

```
// for..of עובר על ערכים במערך או ב iterable
for (const task of tasks) {
  console.log(task.title);
}

// forEach בסדר לתופעות לוואי; לא ניתן לצאת ממנו
tasks.forEach(task => console.log(task.title));

// for קלאסי עדיין שימושי כשהאינדקס חשוב
for (let i = 0; i < tasks.length; i++) {
  console.log(`${i}: ${tasks[i].title}`);
}
```

`for..in` גם קיים אבל עובר על מפתחות של אובייקט, לא על אלמנטים של מערך. מאחר שלמערכים יש מפתחות מחרוזתיים ("0", "1", ...) `for..in` יעשה דברים מפתיעים על מערכים. העדף `for..of` למערכים ו-`Object.keys(obj)` / `Object.entries(obj)` לאובייקטים.

אובייקטים והעתקות רדודות

העתקת אובייקטים היא מקום שמתחילים נשרפים בו. הקצאה לא מעתיקה; היא יוצרת התייחסות חדשה לאותו אובייקט.

```
const original = { title: "Buy milk", done: false };
const alias = original;
alias.done = true;
console.log(original.done); // true – הפתעה
```

כדי להעתיק, השתמש ב-spread (או ב-Object.assign):

```
const copy = { ...original };
copy.done = true;
console.log(original.done); // false עדיין
```

זו העתקה רדודה: אובייקטים מקוננים עדיין משותפים. אם באובייקט שלך יש אובייקטים מקוננים ואתה צריך עותק עמוק, השתמש ב-`structuredClone(value)` (מובנה בדפדפנים מודרניים).

מחלקות, בקצרה

Angular היא class-forward: קומפוננטות, שירותים, דירקטיבות ופיפּים הן כולן מחלקות. אתה לא צריך ללמוד הכל על מחלקות היום – פרק 5 יעשה זאת – אבל התחביר לא מפתיע:

```
class Counter {  
  private count = 0;  
  
  increment(): void {  
    this.count += 1;  
  }  
  
  get value(): number {  
    return this.count;  
  }  
}  
  
const c = new Counter();  
c.increment();  
console.log(c.value); // 1
```

מילת המפתח `private` היא של TypeScript, לא של JavaScript – בזמן ריצה השדה נגיש, אבל הקומפיילר יסרב לתת לקוד חיצוני לגעת בו. ל-JavaScript המודרני יש גם שדות פרטיים אמיתיים `#`; קוד הבסיס של Angular משתמש ב-`private` של TypeScript לפי מוסכמה, אז גם אנחנו.

מה דילגנו עליו במכוון

Generators (`function*`), הטיפוס `Symbol`, `Proxy`, `Reflect`, `Map` ו-`Set` (תפגוש את אלה כשהם עוזרים), ביטויים רגולריים בפירוט, ורבות מהעדינויות של `this`. כולם שימושיים; אף אחד מהם לא נדרש כדי לבנות את `Compass`. כשתפגוש אחד מהם, חפש אותו ב-Mozilla Developer Network – MDN היא הפניה הקנונית ל-JavaScript.

מה בא הלאה

פרק 4 מכניס אותך לאפליקציית Angular רצה. תתקין את Node.js, תתקין את ה-CLI של Angular, תריץ `ng new compass`, ותעבור סיור איטי על כל קובץ שה-CLI מייצר. עד סוף פרק 4, יהיה לך מסך התחלה של Angular פתוח בדפדפן שלך ותבין מה כל חלק ממה שיצר אותו עושה.

תרגילים

1. בהינתן `const nums = [3, 1, 4, 1, 5, 9, 2, 6]` השתמש בשיטות מערך כדי לייצר (א) מערך של ריבועים, (ב) סכום הריבועים, (ג) המספר הגדול ביותר, ו-(ד) האם כל מספר קטן מ-10. כתוב כל אחד כביטוי יחיד.

2. כתוב פונקציה `updateTask(tasks: Task[], id: string, changes: Partial<Task>): Task[]` שמחזירה מערך חדש שבו למשימה עם ה-`id` התואם `changes` ממוזג לתוכה, וכל משימה אחרת נשארת ללא שינוי. השתמש ב-`map` וב-`spread`; אל תשנה את הקלט. (הטיפוס `Partial<Task>` פירושו "כל שדות של `Task`, אבל כל אחד אופציונלי").

3. המר את שרשרת ה-Promise הזאת ל-`await / async`:

```
function loadWithAuth(): Promise<Task[]> {  
  return fetch("/api/session")  
    .then(res => res.json())  
    .then(session => fetch("/api/tasks", { headers: { Authorization: sess  
    .then(res => res.json());  
}
```

אז חשוב מה קורה אם ה- `fetch` הראשון נכשל. איפה השגיאה עולה
על פני השטח בכל גרסה?

פרק 4. ה-CLI, האפליקציה הראשונה שלך, והאנטומיה של פרויקט Angular

הפרק הזה שונה משלושת הקודמים. הוא הפרק הראשון שכדאי לך לקרוא עם המחשב הנייד שלך פתוח. עד סוף הפרק, Compass יתקיים כפרויקט בדיסק שלך, שרת הפיתוח שלו ירוץ, ועמוד ההתחלה שלו יהיה פתוח בדפדפן שלך. ואז נעבור על כל קובץ שה-CLI של Angular ייצר, כך ששום דבר בפרויקט החדש שלך לא ייראה מסתורי בפעם הראשונה שתפתח אותו בעורך.

אם נתקעת בהתקנת Node.js, קפוף לנספח א' להערות ספציפיות לפלטפורמה. חזור כשיהיה לך `node --version` ו-`npm --version` עובדים במסוף שלך.

מה אנחנו מתקינים, ולמה

אתה צריך שלושה דברים כדי לעבוד על Angular:

1. Node.js — סביבת ריצה שמאפשרת ל-JavaScript לרוץ מחוץ לדפדפן. כלי הפיתוח של Angular (הקומפיילר, שרת הפיתוח, מריץ הבדיקות) הם תוכניות JavaScript; הן רצות על Node.js. תתקשר עם Node.js ישירות מעט מאוד, אבל הכל תלוי בו.

2. **npm** – מנהל החבילות של Node.js. הוא מגיע יחד עם Node.js. הוא מוריד ספריות – כולל את Angular עצמה – לתוך תיקייה שנקראת `node_modules` בתוך הפרויקט שלך.

3. **ה-CLI של Angular** – הפקודה `ng`. זה כלי שמייצר תבניות של פרויקטים חדשים, יוצר קבצים חדשים, מריץ את האפליקציה שלך, מריץ את הבדיקות שלך, ובונה להפקה. כל מה שתעשה עם Angular לשארית הספר עובר דרך `ng`.

התקנת Node.js

לך ל-`nodejs.org` והורד את גרסת ה-LTS (long-term support). בזמן כתיבת הספר זו Node.js 20; כשתקרא את זה זה כנראה יהיה Node.js 22 או מאוחר יותר. כל אחת מהן בסדר כל עוד היא 20 ומעלה; Angular מסרבת להתקין על גרסאות ישנות יותר.

עקוב אחרי המתקין לפלטפורמה שלך. ב-macOS וב-Windows, המתקין ממקם את `node` ו-`npm` ב-`PATH` שלך אוטומטית; ב-Linux, השתמש במנהל החבילות של ההפצה שלך או ב-tarball באתר Node.js (ראה נספח א').

פתח מסוף – אפליקציית Terminal ב-macOS, Windows Terminal, או PowerShell ב-Windows, אמולטור המסוף האהוב עליך ב-Linux – ואמת:

```
node --version # או גבוה יותר v20.x.x
npm --version  # או גבוה יותר 10.x.x
```

שניהם צריכים להדפיס מספר גרסה. אם אחד מהם אומר "command not found", ראה נספח א'.

התקנת ה-CLI של Angular

עם Node.js במקומו, התקן את ה-CLI גלובלית עם npm:

```
npm install --global @angular/cli
```

הדגל `--global` הופך את `ng` לזמין בכל מקום במחשב שלך, במקום רק בתיקייה של פרויקט אחד. אמת:

```
ng version
```

אתה אמור לראות את גרסת ה-CLI של Angular, בתוספת כותרת שמפרטת את החבילות שיותקנו. אם אתה מקבל "command not found", פתח חלון מסוף חדש – ה-shell צריך להבחין בבינארי המותקן החדש.

הערה. יש אנשים שמעדיפים אף פעם לא להתקין שום דבר גלובלית. אם זה אתה, אתה יכול להשתמש ב-`npx @angular/` במקום `cli` במקום `ng` בכל מקום בספר הזה. זה מוריד עותק זמני בכל פעם. זה איטי יותר אבל שומר על הסביבה הגלובלית שלך נקייה.

יצירת Compass

בחר תיקייה לשמור בה את הפרויקטים שלך. בספר הזה אשתמש ב-`~/dev`. תחליף בשלך אם אתה מעדיף.

```
cd ~/dev
ng new compass
```

ה-CLI שואל אותך כמה שאלות. תענה עליהן כך:

• **Which stylesheet format would you like to use?** →

`CSS`. לא נשתמש ב-Sass בספר הזה. אם אתה כבר יודע Sass, אתה יכול לבחור בזה; הקוד עדיין יעבוד.

• **...Do you want to enable Server-Side Rendering (SSR)?** →

`No`. נוסף SSR בפרק 18, ברגע שהאפליקציה תהיה שווה `prerender`.

— **Which type of routes would you like to configure?**

אם זה מופיע, בחר בברירת המחדל (`server routes with data prefetching` אם מוצע; אחרת השאר ניתוב מופעל).
CLIs חדשים יותר עושים את זה אוטומטית בלי לשאול.

ה-CLI יוצר תיקיית `compass` לצד המקום שבו הרצת את הפקודה, מוריד כמאתיים חבילות ל-`node_modules` (זה ייקח דקה או שתיים בפעם הראשונה), ועוצר. אז הוא משאיר אותך בפרומפט עם רמז קטן על `ng serve`.

קח את הרמז הזה:

```
cd compass  
ng serve --open
```

כמה שניות אחר כך, הדפדפן שלך יפתח ב-`http://localhost:4200` ותראה את עמוד ההתחלה של Angular ברירת המחדל. ברכות – אתה מריץ אפליקציית Angular.

`ng serve` גם נשאר פתוח במסוף שלך, ומסתכל על קבצי המקור שלך. שמור שינוי, והדפדפן יטען מחדש אוטומטית. השאר את `ng serve` רץ בחלון מסוף אחד; פתח חלון מסוף שני לשארית הפרק.

סיור על כל קובץ שה-CLI ייצר

פתח את `~/dev/compass` בעורך שבחרת. אם אין לך עורך מועדף, התקן את Visual Studio Code מ-`code.visualstudio.com` ופתח את התיקייה עם `File → Open Folder`. ל-VS Code יש את התמיכה הכי טובה מקופסה ב-Angular מכל עורך חינומי; כשנתחיל לכתוב קומפוננטות בפרק 5, הודעות השגיאה שלו `inline` יחסכו לך הרבה זמן.

אנחנו הולכים לצעוד עכשיו דרך כל קובץ וכל תיקייה ברמה העליונה. זה הליכה ארוכה. אל תנסה לשנן; רק תיצור מפה מנטלית של מה כל דבר עושה. תפגוש כל אחד מהקבצים האלה שוב, בהקשר, בהמשך הספר.

`package.json`

זה התיאור של הפרויקט שלך מבחינת `npm`. שני חלקים שווים בדיקה.

`scripts` מפרט את הפקודות שאתה יכול להריץ דרך `npm run` `:<name>`

```
"scripts": {  
  "ng": "ng",  
  "start": "ng serve",  
  "build": "ng build",  
  "watch": "ng build --watch --configuration development",  
  "test": "ng test"  
}
```

`npm run start` ו-`ng serve` שווי ערך. יש קונבנציות שמעדיפות `npm run` כי הוא שומר על אותן פקודות עובדות בפרויקטים שמתמשים בכלים בסיסיים שונים; בפרויקט Angular אתה יכול להשתמש בכל אחד.

`dependencies` מפרט את הספריות שהאפליקציה שלך צריכה בזמן ריצה; `devDependencies` מפרט את הכלים שהיא צריכה במהלך פיתוח. כשגרסה חדשה של Angular יוצאת ואתה רוצה לעדכן, אתה עורך את מספרי הגרסה האלה או מריץ `ng update` – נושא ליום אחר.

`angular.json`

קובץ התצורה של ה-CLI. הוא אומר ל-Angular איך לבנות, להגיש, ולבדוק את הפרויקט. תערוך אותו ידנית לעתים רחוקות בפרקים הראשונים; נבקר בו בפרק 16 (ביצועים) ובפרק 19 (פריסה). לעת עתה, דע שהוא קיים.

tsconfig.json וחבריו

שלושה קבצי תצורת TypeScript:

- **tsconfig.json** – התצורה הבסיסית.
- **tsconfig.app.json** – התצורה שנמצאת בשימוש כשבונים את האפליקציה עצמה.
- **tsconfig.spec.json** – התצורה שנמצאת בשימוש כשמריצים בדיקות.

לא תיגע בהם עד שתתחיל לכתוב בדיקות בפרק 17. הקובץ הראשי מפעיל מצב strict כברירת מחדל בפרויקטי Angular מודרניים, שפירושו ש-TypeScript תהיה קפדנית לגבי בדיקות **any**, **null**, **implicit**, ורגלי-פיתיון אחרות. אל תכבה מצב strict. זו הסיבה שהאפליקציה שלך תישאר ניתנת ל-refactor שנה מהיום.

.editorconfig

קובץ זעיר שמספר לעורך שלך על הזחה, סופי שורה, ובסיסים אחרים. אם העורך שלך מבין אותו (רובם המודרניים מבינים, לפעמים עם תוסף), הפורמט שלך יתאים אוטומטית לברירות המחדל של ה-CLI.

.gitignore

קובץ שמפרט מה git לא צריך לעקוב אחריו. במיוחד הוא לא כולל את **node_modules/** (שהיא ענקית וניתן לשחזר אותה מ-**package.json**) ואת **dist/** (פלט הבנייה). ניצור מאגר git בעוד רגע.

`node_modules/`

החבילות המותקנות. Angular עצמה חיה כאן, יחד עם כל מה ש-Angular תלויה בו ובכל מה שתתקין מאוחר יותר. אל תבדוק את התיקייה הזאת לתוך git; היא לעתים קרובות גיגה-בייטים בגודל וניתן לשחזר אותה במלואה עם `npm install`. אם תמחק אותה, תוכל ליצור אותה מחדש עם `npm install`.

`public/`

קבצים

סטטיים שמוגשים כמו שהם בשורש האפליקציה שלך. אם תפיל `robots.txt` או `favicon.ico` כאן, הם יופיעו ב-`/robots.txt` וב-`/favicon.ico` באפליקציה הרצה. ה-CLI מפיל `favicon.ico` ברירת מחדל כאן.

`src/index.html`

פתח אותו. הוא מאוד קצר:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Compass</title>
  <base href="/">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="icon" type="image/x-icon" href="favicon.ico">
</head>
<body>
  <app-root></app-root>
</body>
</html>
```

האלמנט `<app-root>` הזה הוא נקודת עגון יחידה לכל האפליקציה. כשהדפדפן טוען את הדף, Angular מתאחלת, מוצאת את האלמנט הזה, ומשתלטת עליו. כל מה שאתה רואה ב-`http://localhost:4200` מרונדר בתוך `<app-root>`.

תערוך את `index.html` לעתים רחוקות. תערוך את הכותרת שלו, מדי פעם תוסיף `<link>` לפונט, ותשאיר אותו לבד.

`src/styles.css`

CSS גלובלי. כל מה שיש כאן חל על כל האפליקציה. ייבואי פונטים, reset של CSS, ומאפיינים מותאמים אישית ברמת השורש (משתני CSS) חיים כאן. סגנונות ספציפיים לקומפוננטה – הסגנונות של שורת משימה, נניח – חיים בקובץ הקומפוננטה, שנפגוש בפרק 5.

`src/main.ts`

קובץ ה-`bootstrap`. מאוד קצר:

```
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { App } from './app/app';

bootstrapApplication(App, appConfig)
  .catch(err => console.error(err));
```

`bootstrapApplication` אומר: "התחל את אפליקציית Angular, `App` היא השורשית שלה היא `appConfig`, באיזה אלמנט ב-HTML שיש לו את התג המוגדר על ידי הסלקטור של `App`". התוצאה היא ש-`<app-root>` ב-`index.html` מתמלא בקומפוננטה השורשית המרונדרת.

תערוך את `main.ts` כמעט אף פעם – אולי פעם אחת בפרק 18 כדי לאפשר רינדור בצד השרת.

`src/app/`

זה איפה שהאפליקציה שלך חיה. כל מה שתכתוב מפרק 5 והלאה נכנס ל-`src/app/` או לתת-התיקיות שלו. ה-CLI כבר שם כאן שלושה קבצים:

— `app.ts` (או `app.component.ts` בכמה גרסאות CLI) – הקומפוננטה השורשית. בערך:

```
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet],
  templateUrl: './app.html',
  styleUrls: ['./app.css'],
})
export class App {
  protected readonly title = signal('compass');
}
```

ארבעה דברים שכדאי לשים לב אליהם, שאף אחד מהם אתה לא צריך להבין עדיין:

- הדקורטור `@Component`. הוא אומר ל-Angular שהמחלקה הזאת היא קומפוננטה, ומגדיר את הקומפוננטה עם סלקטור (`<app-root>`), קובץ תבנית, קובץ סגנון, וכל קומפוננטות או דירקטיבות אחרות שהקומפוננטה הזאת משתמשת בתבנית שלה.
- גוף המחלקה. כל דבר שהקומפוננטה רוצה לחשוף לתבנית שלה נכנס כאן – שדות, מתודות, סיגנלים.
- הקריאה `signal(...)`. זה הפרימיטיב של מצב ריאקטיבי מודרני של Angular. התבנית יכולה לקרוא `title()` והיא תרונדר מחדש אם הסיגנל אי פעם ישתנה.
- `imports: [RouterOutlet]`. קומפוננטות עצמאיות מכריזות אילו קומפוננטות ודירקטיבות אחרות הן

משתמשות. RouterOutlet הוא ה-placeholder שבו
ה-router ירנדר עמודים.

כל פרק מפרק 5 והלאה מבלה זמן בתוך קומפוננטות כמו זו. אל
תדאג לפרש אותה עכשיו.

app.html – התבנית של הקומפוננטה השורשית. ה-CLI ממלא
את זה עם עמוד התחלה ממותג של Angular. תמחק את רובו בפרק 5.

app.css – סגנונות scoped ל-קומפוננטה השורשית. כל מה
שתכתוב כאן חל על `<app-root>` ולא על שום דבר אחר – גם אם
יש לך class בשם `.container`. בקובץ הזה ועוד class
בשם `.container`. במקום אחר, הם לא יתנגשו. זה נקרא view
encapsulation, וזה אחד הדברים הטובים בשקט לגבי כתיבת Angular.

app.config.ts – תצורה ברמת האפליקציה.

```
import { ApplicationConfig, provideZoneChangeDetection } from '@angular/core';
import { provideRouter } from '@angular/router';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes),
  ],
};
```

`providers` היא הרשימה של שירותים ותצורה שהאפליקציה שלך
הופכת לזמינים בכל מקום. ב-Angular מודרנית, רוב הדאגות
החוצות-רוחביות מוקמות על ידי קריאה לפונקציות `provideX()`

כאן – ניתוב, HTTP, אנימציות, וכן הלאה. נסיף כמה מהן בפרקים מאוחרים יותר.

app.routes.ts – רשימה ריקה בהתחלה של הגדרות ניתוב:

```
import { Routes } from '@angular/router';

export const routes: Routes = [];
```

פרק 13 ממלא אותה והופך את Compass לאפליקציה רב-עמודית.

מה ה-CLI יכול לעשות מלבד `new` ו-`serve`

תרגל את ההרגל של שימוש ב-`ng generate` לבניית תבניות. זה חוסך הקלדה, ויותר חשוב, זה מציב קבצים חדשים במקום הנכון עם הייבואים הנכונים.

```
ng generate component task-row      # יוצר src/app/task-row/*
ng generate service tasks           # יוצר src/app/tasks.service.ts
ng generate pipe date-friendly     # יוצר src/app/date-friendly.pipe.ts
```

אתה יכול לקצר ל-`ng g c task-row`. בספר הזה תמיד נקליד את הצורה הארוכה כדי שתדע מה אתה עושה; בפרקטיקה יומית תקצר.

פקודות אחרות ששוות לדעת:

```
ng serve      # שרת פיתוח; מסתכל על קבצים, טוען מחדש בשמירה
ng build      # dist בנייה להפקה לתוך /
ng test       # מריץ את בדיקות היחידה
ng lint       # אם מוגדר, linter-מריץ את ה
ng update     # וחבילות קשורות Angular מעדכן את
```

Git, ברגע זה

לפני שאתה כותב שורת קוד יחידה, יצור מאגר git. האני העיתידי שלך יודה לך בפעם הראשונה שתשבור משהו ותרצה לראות מה היה לך לפני חמש דקות.

```
git init
git add .
git commit -m "Initial commit from ng new"
```

אם מעולם לא השתמשת ב-git, בלה עשרים דקות עם שני הפרקים הראשונים של כל מדריך git ראשוני (git-scm.com/book) הוא חינם). תשתמש ב- `git status`, `git add`, `git commit`, ו-`git diff` בערך חמישים פעם כל אחד בספר הזה.

לוודא שהכל עובד

עם `ng serve` עדיין רץ מקודם, פתח את `src/app/app.html` בעורך שלך. מצא את ה- `<h1>` הראשון (או מקום עם טקסט גלוי) ושנה טקסט כלשהו ל- `Hello, Compass.` שמור את הקובץ.

טאב הדפדפן שלך אמור להתרענן תוך שנייה, והטקסט החדש אמור להופיע. אם כן, יש לך סביבת פיתוח Angular עובדת.

אם לא – העמוד לא מתעדכן, או שהמסוף מדפיס שגיאה – ראה את סעיף פתרון הבעיות בנספח א', או את נספח ב', שהוא מדריך לקריאת הודעות שגיאה של Angular.

מה בא הלאה

חלק שני. בפרק 5 נמחק את רוב תוכן ההתחלה של ה-CLI ונבנה את הקומפוננטה האמיתית הראשונה של Compass: רשימה של משימות. תפגוש תבניות, קישורי נתונים, ואת תחביר בקרת הזרימה החדש. עד סוף פרק 5, Compass תהיה רשימת מטלות – פשוטה מאוד, רצה כולה בדפדפן, בלי כל השמירה, הניתוב או האימות של המוצר המוגמר. כל פרק עוקב מוסיף עוד יכולת אמיתית אחת.

תרגילים

1. הרץ `ng generate component about --skip-tests` וקרא את הפלט. הדגל `--dry-run` אומר ל-CLI להדפיס מה הוא היה עושה בלי לעשות זאת בפועל. אילו קבצים היו נוצרים? עשה את אותו הדבר עבור `ng generate service auth --dry-run`. זה הרגל שימושי לתצוגה מקדימה של מה ה-CLI רוצה לעשות לפני שהוא עושה זאת.

2. פתח את כלי הפיתוח של הדפדפן (F12 ברוב הדפדפנים) ולחץ

על טאב "Elements" או "Inspector". מצא את האלמנט `<app-root>` ב-DOM. שים לב שהוא אלמנט אמיתי בעמוד, והתוכן של האפליקציה מרונדר בתוכו. זה שווה לראות פעם אחת כדי שתדע ש-Angular לא עושה שום דבר קסום ל-HTML.

3. ב-`src/app/app.html`, שים `{{ title() }}` איפשהו שהטקסט יהיה גלוי, ושמור. שים לב לסוגריים אחרי `title` – זה לא טעות הקלדה. סיגנלים הם פונקציות שאתה קורא להן כדי לקבל את הערך הנוכחי שלהן. בפרק 7 נבחן למה. לעת עתה, פשוט תבחין שזה עובד.